



ArangoDB

(<https://www.arangodb.com/>)



3.10 ▾

[3.10.14 \(/3.10/\)](#)



[Data Science \(/3.10/data-science/\)](#)



[Pregel \(/3.10/data-science/pregel/\)](#)



Ask AI



ArangoDB v3.10 reached End of Life (EOL) and is no longer supported.



This documentation is outdated.
Please see the most recent [stable version \(/stable/\)](/stable/).

Distributed Iterative Graph Processing (Pregel)

Pregel enables you to do online analytical processing directly on graphs stored in ArangoDB

Distributed graph processing enables you to do online analytical processing directly on graphs stored in ArangoDB. This is intended to help you gain analytical insights on your data, without having to use external processing systems. Examples of algorithms to execute are PageRank, Vertex Centrality, Vertex Closeness, Connected



ArangoDB

(<https://www.arangodb.com/>)

Components, Community Detection. For more details, see all [available algorithms \(/3.10/data-science/pregel/algorithms/\)](/3.10/data-science/pregel/algorithms/) in ArangoDB.

Check out the hands-on [ArangoDB Pregel Tutorial \(https://www.arangodb.com/pregel-community-detection/\)](https://www.arangodb.com/pregel-community-detection/) [↗](#) to learn more.

The processing system inside ArangoDB is based on: [Pregel: A System for Large-Scale Graph Processing \(http://www.dcs.bbk.ac.uk/~dell/teaching/cc/paper/sigmod10/p135-malewicz.pdf\)](http://www.dcs.bbk.ac.uk/~dell/teaching/cc/paper/sigmod10/p135-malewicz.pdf) [↗](#) – Malewicz et al. (Google), 2010. This concept enables us to perform distributed graph processing, without the need for distributed global locking.

This system is not useful for typical online queries, where you just work on a small set of vertices. These kind of tasks are better suited for [AQL traversals \(/3.10/aql/graphs/\)](/3.10/aql/graphs/).

Prerequisites

If you run a single ArangoDB instance in single-server mode, there are no requirements regarding the modeling of your data. All you need is at least one vertex collection and one edge collection.

In cluster deployments, the collections need to be sharded in a specific way to ensure correct results: The outgoing edges of a vertex need to be on the same DB-Server as the vertex. This is guaranteed by SmartGraphs (/3.10/graphs/smartgraphs/). Thus, Pregel in cluster deployments is not usable in the Community Edition.

Note that the performance may be better, if the number of your shards / collections matches the number of CPU cores.

JavaScript API

Starting an Algorithm Execution

The Pregel API is accessible through the `@arangodb/pregel` package.

To start an execution, you need to specify the **algorithm** name and a named graph (SmartGraph in cluster). Alternatively, you can specify the vertex and edge collections. Additionally, you can specify custom parameters which vary for each

 [CONTRIBUTE](#)

[On this page](#)
([HTTPS://GITHUB.COM/ARANGODB/DOCS-](https://github.com/arangodb/docs-HUGO/EDIT/MAIN/SITE/CONTENT/3.10/DATA-JavaScript%20API)
Prerequisites
[HUGO/EDIT/MAIN/SITE/CONTENT/3.10/DATA-JavaScript API](#)

[Starting an Algorithm Execution](#)
([SCIENCE/PREGEL/_INDEX.MD](#))

Status of an Algorithm Execution

Canceling an Execution / Discarding results

[AQL integration](#)

[Algorithm Parameters](#)

[Limits](#)



algorithm. The `start()` method always returns a unique ID (a numeric string) which you can use to interact with the algorithm later on.

The following example shows the `start()` method variant for using a named graph:

```
var pregel = require("@arangodb/pregel");
var params = {};
var execution = pregel.start("<algorithm>", "
<graphname>", params);
```

You can also specify the vertex and edge collections directly. In this case, the second argument must be an object with the keys `vertexCollections` and `edgeCollections`:

```
var execution = pregel.start("<algorithm>", {
  vertexCollections: ["vertices"], edgeCollections:
  ["edges"] }, params);
```

The `params` argument needs to be an object with the algorithm settings as described in [Pregel Algorithms \(/3.10/data-science/pregel/algorithms/\)](https://www.arangodb.com/docs/stable/data-science/pregel/algorithms/).

Status of an Algorithm Execution

You can call `pregel.status()` and use the ID returned by the `pregel.start(...)` method to track the status of your algorithm:

```
var execution = pregel.start("sssp", "demograph",  
  { source: "vertices/V" });  
var status = pregel.status(execution);
```

It tells you the current `state` of the execution, the current global superstep, the runtime, the global aggregator values as well as the number of send and received messages.

The `state` field has one of the following values:

State	Description
<code>"none"</code>	The Pregel run has not started yet.
<code>"loading"</code>	The graph data is being loaded from the database into memory before executing the algorithm.
<code>"running"</code>	The algorithm is executing normally.
<code>"storing"</code>	The algorithm finished, but the results are still being written back into the collections. Only occurs if the <code>store</code> parameter is set to <code>true</code> .



ArangoDB

(<https://www.arangodb.com/>)

State	Description
"done"	The execution is done. In version 3.7.1 and later, this means that storing is also done. In earlier versions, the results may not be written back into the collections yet. This event is announced in the server log (requires at least the <code>info</code> log level for the <code>pregel</code> log topic).
"canceled"	The execution was permanently canceled, either by the user or by an error.
"in error"	The execution is in an error state. This can be caused by primary DB-Servers being unreachable or unresponsive. The execution might recover later, or switch to <code>canceled</code> if it is not able to recover successfully.
"recovering"	The execution is actively recovering and switches back to <code>running</code> if the recovery is successful.
"fatal error"	The execution resulted in an non-recoverable error.

The object returned by the `status()` method looks like this:

```
{
  "state" : "running",
  "gss" : 12,
  "totalRuntime" : 123.23,
  "aggregators" : {
    "converged" : false,
    "max" : true,
    "phase" : 2
  },
  "sendCount" : 3240364978,
  "receivedCount" : 3240364975
}
```

Canceling an Execution / Discarding results

To cancel an execution which is still running and discard any intermediate results, you can use the `pregel.cancel()` method. This immediately frees all memory taken up by the execution, and makes you lose all intermediary data.

```
// start a single source shortest path job  
var execution = pregel.start("sssp", "demograph",  
  { source: "vertices/V" });  
pregel.cancel(execution);
```

You might get inconsistent results if you requested to store the results and then cancel an execution when it is already in its `storing` state (or `done` state in versions prior to 3.7.1). The data is written multi-threaded into all collection shards at once. This means there are multiple transactions simultaneously. A transaction might already be committed when you cancel the execution job. Therefore, you might see some updated documents, while other documents have no or stale results from a previous execution.

When the graph processing subsystem finishes executing an algorithm, the results can either be written back into documents (using `store: true` as a parameter) or kept in memory only (using `store: false`). If the data is persisted, then you can query the documents normally to get access to the results.

If you do not want to store results, then they are only held temporarily, until you call the `cancel()` method, or their time to live (customizable via the `ttl` parameter) is exceeded. The in-memory results can be accessed via the `PREGEL_RESULT()` AQL function. If the results are stored in documents, they are not queryable by `PREGEL_RESULT()`.

The result field names depend on the algorithm in both cases.

For example, you might want to query only nodes with the highest rank from the result set of a PageRank execution:



```
FOR v IN PREGEL_RESULT(<jobId>)
  FILTER v.result >= 0.01
  RETURN v._key
```

By default, the `PREGEL_RESULT()` AQL function returns the `_key` of each vertex plus the result of the computation. In case the computation was done for vertices from different vertex collection, just the `_key` values may not be sufficient to tell vertices from different collections apart. In this case, `PREGEL_RESULT()` can be given a second parameter `withId`, which makes it return the `_id` values of the vertices as well:

```
FOR v IN PREGEL_RESULT(<jobId>, true)
  FILTER v.result >= 0.01
  RETURN v._id
```

Algorithm Parameters

There are a number of general parameters which apply to almost all algorithms:

- `store` (bool): Defaults to `true`. If enabled, the Pregel engine writes results back to the database. If the value is `false`, then you can query the results with `PREGEL_RESULT()` in AQL. See [AQL integration](#).
- `maxGSS` (number): Maximum number of global iterations for this algorithm
- `parallelism` (number): Number of parallel threads to use per worker. Does not influence the number of threads used to load or store data from the database (this depends on the number of shards).
- `async` (bool): If enabled, algorithms which support an asynchronous mode run without synchronized global iterations. Might lead to performance increases if you have load imbalances.
- `resultField` (string): Most algorithms use this as attribute name for the result. Some use it as prefix for multiple result attributes. Defaults to `"result"`.
- `useMemoryMaps` (bool): Use disk based files to store temporary results. This might make the computation disk-bound, but allows you to run computations which would not fit into main memory. It is recommended to set this flag for larger datasets.
- `shardKeyAttribute` (string): shard key that edge collections are sharded after (default: `"vertex"`)
- `ttl` (number): The time to live (TTL) defines for how long (in seconds) the Pregel run is

Limits

Pregel algorithms in ArangoDB store temporary vertex and edge data in main memory by default. For large datasets, this can cause problems, as servers may run out of memory while loading the data.

To avoid running out of memory, you can start Pregel jobs with the `useMemoryMaps` attribute set to `true`. This makes the algorithms use memory-mapped files as a backing storage in case of huge datasets. Falling back to memory-mapped files might make the computation disk-bound, but may be the only way to complete the computation at all.

Parts of the Pregel temporary results (aggregated messages) may also be stored in the main memory, and currently the aggregation cannot fall back to memory-mapped files. That means, if algorithms need to store a lot of result messages temporarily, they may consume a lot of the main memory.

In general, it is also recommended to set the `store` attribute of Pregel jobs to `true`, to make jobs write the results back to disk and not just hold them in the main memory. This way, the results are removed from the main memory once



ArangoDB

(<https://www.arangodb.com/>)

a Pregel job completes. If the `store` attribute is explicitly set to `false`, result sets of completed Pregel runs are not removed from main memory until you explicitly discard them by calling the `cancel()` method (or shutting down the server).

< Large Language Models and Knowledge Graphs (/3.10/data-science/llm-pregel-knowledge-graphs/) Algorithms (/3.10/data-science/pregel/algorithms/)